UNIVERSITY OF HERTFORDSHIRE
School of Physics, Engineering and Computer Science

Final Project Report (FPR)

7PAM2002-0509: MSc Data Science Project
22-April-2026

"Smart Energy consumption Forecasting and Anomaly Detection Using Time-Series Machine Learning Models"

Name: Mohan Marthala
Student ID: 24068622
Supervisor: Vid Irsic

Abstract

This study investigates the application of time-series machine learning models for forecasting household electricity consumption and detecting anomalous usage patterns using the Individual Household Electric Power Consumption dataset. The main aim is to evaluate how accurately future energy consumption can be predicted and whether abnormal electricity usage can be automatically identified. To achieve this, the dataset was preprocessed and enhanced through feature engineering, and three forecasting models, RandomForest, XGBoost, and Prophet, were implemented and evaluated using Root Mean Square Error (RMSE) and Mean Absolute Error (MAE). In addition to forecasting, three anomaly detection methods—rolling Z-score, Isolation Forest, and Prophet residual analysis—were applied to identify unusual energy consumption behaviour. The results showed that RandomForest achieved the best forecasting performance with the lowest error values, followed by XGBoost, while Prophet was effective in modelling trend and seasonality but produced comparatively higher prediction errors. The anomaly detection methods successfully identified abnormal energy usage patterns, with each method capturing different types of anomalies. Overall, the findings demonstrate that combining machine learning-based forecasting with anomaly detection provides a comprehensive and effective approach for analysing household energy consumption and supporting smarter energy management.

Acknowledgement

I would like to express my deepest gratitude to my supervisor, Vid Irsic, for their guidance, insightful feedback, and encouragement throughout the course of this research. I am equally grateful to my professors and peers for their constructive suggestions and to my family for their unconditional support and motivation. Special thanks are also extended to the providers of the open-source weather datasets, without which this study would not have been possible.

MSc Final Project Declaration

This report is submitted in partial fulfilment of the requirement for the degree of Master of Science **in Data Science** at the University of Hertfordshire.

I have read the detailed guidance to students on academic integrity, misconduct and plagiarism information at Assessment Offences and Academic Misconduct and understand the University process of dealing with suspected cases of academic misconduct and the possible penalties, which could include failing the project or course.

I certify that the work submitted is my own and that any material derived or quoted from published or unpublished work of other persons has been duly acknowledged. (Ref. UPR AS/C/6.1, section 7 and UPR AS/C/5, section 3.6)

I did not use human participants in my MSc Project.

I hereby give permission for the report to be made available on module websites provided the source is acknowledged.

Student Name printed: Mohan Marthala

Student SRN number: 24068622

Table of Contents

Abstract 2
Acknowledgement 3
MSc Final Project Declaration 4
Chapter 1 – Introduction 7
1.1 Background 7
1.2 Problem Statement 7
1.3 Aim and Objectives 8
1.4 Research Question 8
1.5 Scope of the Study 8
Chapter 2 – Literature Review 9
2.1 Energy Consumption Forecasting 9
2.1 Machine Learning in Energy Prediction 9
2.2 Prophet and Modern Forecasting Approaches 10
2.3 Anomaly Detection in Energy Data 10
2.4 Research Gap 11
Chapter 3 – Methodology 12
3.1 Research Design 12
3.2 Dataset Description 12
3.3 Data Preprocessing 12
3.3.1 Handling Missing Values 12
3.3.2 Data Type Conversion 12
3.3.3 Datetime Processing 13
3.3.4 Resampling 13
3.4 Feature Engineering 13
3.4.1 Time-Based Features 13
3.4.2 Lag Features 13
3.4.3 Rolling Statistics 13
3.5 Forecasting Models 13
3.5.1 RandomForest 13
3.5.2 XGBoost 14
3.5.3 Prophet 14
3.6 Train-Test Split 14
3.7 Evaluation Metrics 14
3.8 Anomaly Detection Methods 14
3.8.1 Rolling Z-Score 14

3.8.2 Isolation Forest 14
3.8.3 Prophet Residual Analysis 15
3.9 Ethical Considerations 15
Chapter 4 – Results and Discussion 16
4.1 Exploratory Data Analysis 16
4.1.1 Time-Series Behaviour of Global Active Power 16
4.1.2 Distribution of Global Active Power 16
4.1.3 Average Electricity Usage by Hour of Day 17
4.1.4 Average Electricity Usage by Day of Week 18
4.1.5 Correlation Analysis After Feature Engineering 18
4.2 Forecasting Results 19
4.2.1 RandomForest Forecasting Performance 19
4.2.2 XGBoost Forecasting Performance 19
4.2.3 Prophet Forecasting Performance 19
4.2.4 Model Comparison 20
4.2.5 Visual Analysis of Forecasting Models 20
4.3 Anomaly Detection Results 23
4.3.1 Comparison of Anomaly Detection Methods 24
4.3.2 Visual Analysis of Anomaly Detection Results 24
4.4 Discussion of Findings 27
Chapter 5 – Conclusion and Future Work 28
5.1 Summary of the Study 28
5.2 Key Findings 28
5.3 Research Contributions 28
5.4 Limitations of the Study 29
5.5 Future Work 29
5.6 Conclusion 30
References 31
Appendix 32

Chapter 1 – Introduction

1.1 Background

The increasing demand for electricity and the growing complexity of modern energy systems have made energy consumption forecasting an important research area in both academia and industry. Accurate forecasting of electricity usage supports efficient energy generation, distribution, and management, while also contributing to sustainability and cost reduction. Household-level electricity consumption, in particular, is highly variable and influenced by factors such as occupant behaviour, appliance usage, daily routines, and seasonal changes, making it a challenging forecasting problem (Cascone *et al*., 2023). As smart metering technologies become more widespread, large volumes of time-stamped energy data are now available, creating opportunities to apply machine learning and time-series forecasting techniques for improved energy analysis and prediction.

Traditional statistical approaches such as the Autoregressive Integrated Moving Average (ARIMA) model have long been used for time-series forecasting, especially where data exhibit structured temporal patterns (Ahmed and Khamees, 2025). However, such methods often struggle to capture nonlinear and highly dynamic patterns that are common in real-world household energy consumption data. More recently, machine learning and deep learning approaches have gained popularity due to their ability to model complex relationships and extract useful predictive patterns from historical data (Syed *et al*., 2021). Tree-based models such as RandomForest and XGBoost, as well as specialised forecasting tools such as Prophet, have demonstrated strong performance in time-series prediction tasks due to their flexibility, robustness, and ability to handle temporal variation.

In addition to forecasting future energy consumption, the identification of abnormal or anomalous electricity usage is also an important challenge. Unusual consumption patterns may indicate inefficient appliance use, faulty equipment, unexpected occupant behaviour, or energy wastage. Detecting such anomalies can support more intelligent energy management and contribute to early intervention strategies. Therefore, combining forecasting with anomaly detection offers a more comprehensive understanding of energy usage behaviour than prediction alone. This study addresses both aspects by implementing forecasting models and anomaly detection techniques on household electricity consumption data.

1.2 Problem Statement

Forecasting household electricity consumption is a difficult task due to the highly variable and nonlinear nature of energy usage patterns. Existing forecasting methods often focus primarily on predicting future values but may not adequately address abnormal energy usage behaviour. Furthermore, while advanced deep learning approaches have shown promising results in recent studies, they can be computationally expensive and difficult to interpret, particularly for smaller-scale academic projects or real-world deployment (Cascone *et al*., 2023; Syed *et al*., 2021). There is therefore a need to investigate whether more accessible machine learning and time-series models can provide accurate household energy forecasting while also supporting anomaly detection.

Another challenge lies in understanding how different forecasting models compare when applied to the same dataset and whether they can effectively support energy management

tasks. In addition, abnormal energy usage events are often overlooked in standard forecasting studies, despite their practical importance. This project seeks to address these issues by evaluating multiple forecasting models and integrating anomaly detection techniques into the analysis.

1.3 Aim and Objectives

This project aims to develop and evaluate time-series machine learning models for forecasting household electricity consumption and detecting anomalous usage patterns.

To achieve this aim, the following objectives were defined:

- To preprocess and explore the Individual Household Electric Power Consumption dataset.
- To engineer relevant time-based and lag-based features for forecasting.
- To implement and compare the performance of RandomForest, XGBoost, and Prophet models for electricity consumption forecasting.
- To evaluate forecasting performance using appropriate regression metrics, including RMSE and MAE.
- To apply anomaly detection methods, including rolling Z-score, Isolation Forest, and Prophet residual analysis, to identify abnormal energy usage patterns.
- To analyse the forecasting and anomaly detection results and assess their usefulness for smart energy management.

1.4 Research Question

The following research question guides this project:

How accurately can household electricity consumption be predicted using time-series models, and can machine learning methods automatically detect abnormal energy usage patterns?

1.5 Scope of the Study

This study focuses on household-level electricity consumption forecasting using a publicly available benchmark dataset from the UCI Machine Learning Repository. The dataset contains historical energy consumption measurements collected from a single household in France between 2006 and 2010. The analysis is limited to numerical time-series variables related to electricity usage, such as global active power, voltage, and sub-metering values.

The forecasting component of the study is restricted to three selected models: RandomForest, XGBoost, and Prophet. These models were chosen because they represent different forecasting approaches, including ensemble machine learning and dedicated time-series modelling. The anomaly detection component is limited to three methods: rolling Z-score, Isolation Forest, and Prophet residual analysis. While other advanced deep learning methods, such as LSTM or Transformer-based models, could also be applied, they are considered outside the scope of the current study and are suggested as future work.

Chapter 2 – Literature Review

2.1 Energy Consumption Forecasting

Energy consumption forecasting has become an important area of research due to its relevance to smart grids, sustainable energy management, and efficient resource planning. Accurate forecasting helps energy providers and consumers better understand demand patterns, reduce waste, and improve the allocation of energy resources. Household-level forecasting is particularly challenging because energy usage is highly dependent on behavioural, temporal, and environmental factors, which often create irregular and nonlinear consumption patterns (Syed *et al*., 2021).

Research in this area has shown that electricity demand forecasting is valuable for balancing supply and demand, improving load scheduling, and supporting energy-saving strategies. Cascone *et al*. (2023) highlighted that household electricity forecasting is increasingly important in smart and sustainable environments, particularly where smart metering devices continuously collect energy consumption data. Their study also emphasised that accurate demand prediction is useful not only for planning and distribution but also for understanding the dynamic nature of household consumption behaviour.

At the household level, forecasting is more difficult than at aggregated levels such as buildings, neighbourhoods, or cities, because household usage is less stable and more sensitive to individual activities and routines. This variability makes household energy forecasting a challenging but valuable problem for machine learning and time-series modelling.

2.1 Machine Learning in Energy Prediction

Machine learning has become increasingly important in energy consumption forecasting because of its ability to model nonlinear patterns and interactions between multiple input variables. Tree-based ensemble methods such as RandomForest and gradient boosting techniques have been widely applied in forecasting problems because of their robustness and predictive power. These models are capable of handling large datasets, nonlinear relationships, and feature interactions effectively. In energy forecasting contexts, machine learning approaches are often used alongside feature engineering techniques such as lag variables, rolling averages, and time-based attributes to improve prediction performance.

Syed *et al*. (2021) discussed the limitations of simpler machine learning methods in highly volatile energy consumption data and proposed a hybrid deep learning model for household-level forecasting in smart buildings. Their study demonstrated that deep learning architectures could outperform traditional methods in certain settings, particularly when long-term dependencies and nonlinear patterns are present. However, such models can also be computationally demanding and require more complex tuning and interpretation.

Cascone *et al*. (2023) also showed that deep learning and hybrid architectures such as LSTM and ConvLSTM are effective for multi-step household electricity forecasting. Their study reported strong predictive performance using a hybrid ConvLSTM architecture and highlighted the growing importance of artificial intelligence methods in smart energy applications. While deep learning approaches have shown promising results, simpler machine learning models such as

RandomForest and XGBoost remain attractive in academic and practical settings due to their efficiency, interpretability, and competitive performance.

2.2 Prophet and Modern Forecasting Approaches

In addition to classical time-series methods and machine learning models, modern forecasting tools such as Prophet have gained popularity for handling time-series data with trend and seasonality. Prophet was developed to provide a flexible and user-friendly forecasting framework capable of modelling nonlinear trends, daily and weekly seasonality, and holiday effects. It is particularly useful when the primary forecasting objective is to capture interpretable temporal patterns rather than only maximise predictive performance.

Although Prophet is not as commonly used in household energy forecasting literature as ARIMA or deep learning models, it is relevant for this study because household electricity consumption often exhibits strong daily and weekly seasonal cycles. Prophet therefore offers a useful comparison against machine learning approaches such as RandomForest and XGBoost, particularly in terms of its ability to model structured time-series behaviour and support residual-based anomaly detection.

2.3 Anomaly Detection in Energy Data

While many studies focus primarily on forecasting future electricity consumption, fewer studies address the detection of abnormal energy usage patterns. Anomaly detection is an important aspect of energy analytics because unusual electricity consumption may indicate equipment faults, inefficient appliance usage, irregular occupancy behaviour, or unexpected operational events. Identifying such anomalies can help improve energy efficiency, support fault diagnosis, and contribute to intelligent monitoring systems.

Anomaly detection methods can be broadly divided into statistical, machine learning, and model-based approaches. Statistical techniques such as Z-score and moving-window deviation analysis are commonly used because they are simple to implement and effective at detecting extreme values. However, these methods may not fully capture multivariate or context-dependent anomalies.

Machine learning-based anomaly detection methods, such as Isolation Forest, offer greater flexibility by identifying unusual patterns across multiple features rather than relying only on threshold-based deviations. Isolation Forest is particularly useful for high-dimensional data because it isolates anomalies by recursively partitioning the feature space. In time-series applications, model-based anomaly detection can also be performed by analysing forecasting residuals. If the difference between the predicted value and the actual observation is unusually large, the event may be considered anomalous. This makes forecasting models such as Prophet useful not only for prediction but also for residual-based anomaly identification.

Despite the practical importance of anomaly detection, many household energy forecasting studies focus only on prediction performance and do not integrate anomaly analysis into the modelling pipeline. This suggests a research opportunity that combines both forecasting and anomaly detection within the same framework.

2.4 Research Gap

The reviewed literature shows that considerable progress has been made in household energy forecasting using statistical models, machine learning methods, and deep learning architectures. Studies such as Cascone *et al*. (2023) and Syed *et al*. (2021) demonstrate the effectiveness of advanced forecasting techniques, particularly hybrid deep learning models, in capturing household energy consumption patterns. Similarly, Ahmed and Khamees (2025) illustrate the continued relevance of statistical time-series methods for structured energy forecasting tasks.

However, several gaps remain. First, many studies focus primarily on forecasting accuracy without incorporating anomaly detection, despite the practical importance of identifying unusual energy usage. Second, although deep learning methods often achieve strong performance, they can be computationally expensive and difficult to interpret, making them less accessible for smaller-scale or practical implementations. Third, there is value in comparing multiple forecasting approaches—including ensemble machine learning models and dedicated time-series tools—within a single study to better understand their relative strengths and weaknesses.

This project addresses these gaps by combining forecasting and anomaly detection in a unified framework using the Individual Household Electric Power Consumption dataset. It compares RandomForest, XGBoost, and Prophet for forecasting performance while also applying rolling Z-score, Isolation Forest, and Prophet residual analysis for anomaly detection. In doing so, the study contributes a more comprehensive analysis of household electricity usage behaviour.

Chapter 3 – Methodology

3.1 Research Design

This study follows a quantitative, data-driven research approach using time-series data. The research is experimental in nature, where multiple machine learning and time-series models are implemented and evaluated on the same dataset. The workflow includes:

1. Data collection and preprocessing
2. Exploratory data analysis (EDA)
3. Feature engineering
4. Model training and evaluation
5. Anomaly detection
6. Comparative analysis of results

The models are trained using historical data and evaluated on unseen future data to simulate real-world forecasting scenarios. This approach ensures that the models are assessed based on their ability to generalise to new data.

3.2 Dataset Description

The dataset used in this study is the Individual Household Electric Power Consumption dataset, obtained from the UCI Machine Learning Repository. It contains measurements of household electricity consumption recorded over a period from December 2006 to November 2010.

The dataset consists of time-series data recorded at one-minute intervals, including several electrical variables such as global active power, voltage, global intensity, and sub-metering values. Due to its high temporal resolution and long duration, the dataset is suitable for both forecasting and anomaly detection tasks.

The data is provided in a text format (.txt) with semicolon-separated values, which was loaded and processed using Python-based data analysis libraries.

3.3 Data Preprocessing

Data preprocessing is an essential step to ensure data quality and prepare the dataset for modelling. Several preprocessing steps were applied in this study:

3.3.1 Handling Missing Values

The dataset contains missing values represented by non-numeric entries. These values were converted into standard missing value indicators and removed to ensure that the dataset contained only valid numerical observations.

3.3.2 Data Type Conversion

All relevant variables, including global active power, voltage, and sub-metering values, were converted into numerical data types to enable mathematical operations and model training.

3.3.3 Datetime Processing

The separate date and time columns were combined to create a unified datetime variable. This variable was then set as the index of the dataset to support time-series analysis.

3.3.4 Resampling

The original dataset contained minute-level observations, which can be noisy and computationally intensive. Therefore, the data was **resampled to hourly intervals** by computing the mean of each hour. This step reduced noise and improved computational efficiency while preserving overall consumption patterns.

3.4 Feature Engineering

Feature engineering was performed to enhance the predictive capability of the models by incorporating temporal and historical information.

3.4.1 Time-Based Features

Two time-based features were extracted:

- **Hour of the day**, to capture daily consumption patterns
- **Day of the week**, to capture weekly behavioural trends

3.4.2 Lag Features

Lag features were created to include past consumption values as inputs to the models:

- **Lag 1** (previous hour consumption)
- **Lag 24** (same hour on the previous day)

These features help the models learn temporal dependencies in the data.

3.4.3 Rolling Statistics

A **24-hour rolling mean** was calculated to smooth short-term fluctuations and capture longer-term consumption trends.

These engineered features improve the model's ability to predict future electricity consumption by incorporating both temporal patterns and historical context.

3.5 Forecasting Models

Three models were selected to represent different forecasting approaches:

3.5.1 RandomForest

RandomForest is an ensemble learning algorithm based on multiple decision trees. Each tree is trained on a random subset of the data, and the final prediction is obtained by averaging the outputs of all trees. This approach reduces overfitting and improves generalisation.

RandomForest was used in this study because of its ability to handle nonlinear relationships and complex feature interactions in time-series data.

3.5.2 XGBoost

XGBoost is a gradient boosting algorithm that builds decision trees sequentially. Each new tree attempts to correct the errors of the previous trees, leading to improved prediction accuracy.

XGBoost is known for its high performance, scalability, and ability to handle structured data effectively. It was included in this study to compare its performance with RandomForest.

3.5.3 Prophet

Prophet is a time-series forecasting model designed to capture trend and seasonality in temporal data. It models time-series data using additive components, including trend, seasonal effects, and residuals.

Prophet was selected because household electricity consumption exhibits strong daily and weekly patterns, which can be effectively modelled using this approach.

3.6 Train-Test Split

Since this study involves time-series data, a **chronological train-test split** was used instead of random sampling. The dataset was divided into:

- **80% training data** (past observations)
- **20% testing data** (future observations)

3.7 Evaluation Metrics

The performance of the forecasting models was evaluated using the following regression metrics:

- **Root Mean Square Error (RMSE):** Measures the magnitude of prediction errors and penalises larger errors more heavily.
- **Mean Absolute Error (MAE):** Measures the average absolute difference between predicted and actual values.

These metrics provide a reliable assessment of model accuracy for continuous prediction tasks.

3.8 Anomaly Detection Methods

In addition to forecasting, three anomaly detection techniques were applied to identify abnormal energy consumption patterns.

3.8.1 Rolling Z-Score

The rolling Z-score method calculates how far a data point deviates from the rolling mean, measured in terms of standard deviations. Observations with a Z-score greater than a predefined threshold (e.g., ±3) are classified as anomalies.

3.8.2 Isolation Forest

Isolation Forest is a machine learning-based anomaly detection algorithm that isolates anomalies by randomly partitioning the data. Anomalies are more easily isolated than normal points and are therefore identified based on shorter path lengths in the decision trees.

3.8.3 Prophet Residual Analysis

In this method, anomalies are detected based on the difference between actual values and predicted values from the Prophet model. Large residuals indicate unexpected deviations from the forecasted pattern and are classified as anomalies using a statistical threshold.

3.9 Ethical Considerations

The dataset used in this study is publicly available and anonymised, ensuring that no personally identifiable information is included. However, electricity consumption data may still reveal behavioural patterns of household occupants. Therefore, the data is used strictly for research purposes, and care is taken to ensure responsible and ethical use of the information.

Chapter 4 – Results and Discussion

4.1 Exploratory Data Analysis

Exploratory Data Analysis (EDA) was conducted to understand the characteristics of the household electricity consumption dataset and identify patterns that could influence forecasting performance. The analysis focused on overall consumption behaviour, statistical distribution, daily and weekly trends, and relationships between variables.

4.1.1 Time-Series Behaviour of Global Active Power

Figure 4.1 illustrates the hourly time-series of Global Active Power, highlighting clear fluctuations in household electricity consumption over the study period. The data shows varying energy usage with noticeable peaks and troughs, including several high-consumption spikes exceeding 6 kW, likely due to heavy appliance use or unusual activity. Recurring patterns across different periods suggest the presence of seasonal and behavioural trends, indicating strong temporal structure in the dataset. This supports its suitability for time-series forecasting and the application of anomaly detection to identify irregular consumption patterns.

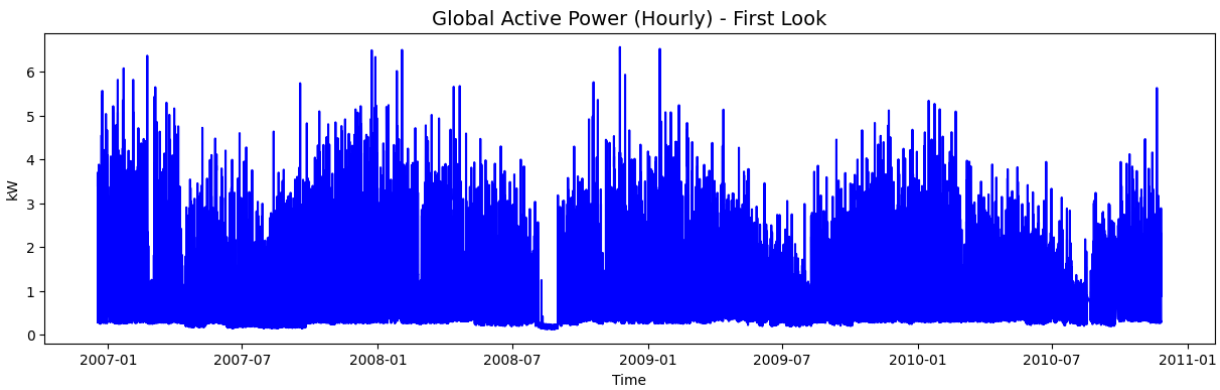


Figure 4.1: Hourly time-series plot of Global Active Power.

4.1.2 Distribution of Global Active Power

Figure 4.2 presents the distribution of the Global Active Power variable, showing a positively skewed (right-skewed) pattern where lower electricity consumption values are more frequent and higher values are relatively rare. Most observations are concentrated below approximately 2 kW, with a long tail extending beyond 6 kW, reflecting typical residential energy usage with occasional high-demand periods. This skewness indicates that forecasting models must account for both common low-consption behaviour and infrequent high-consumption spikes, while also highlighting the importance of anomaly detection to identify unusual or extreme usage events.

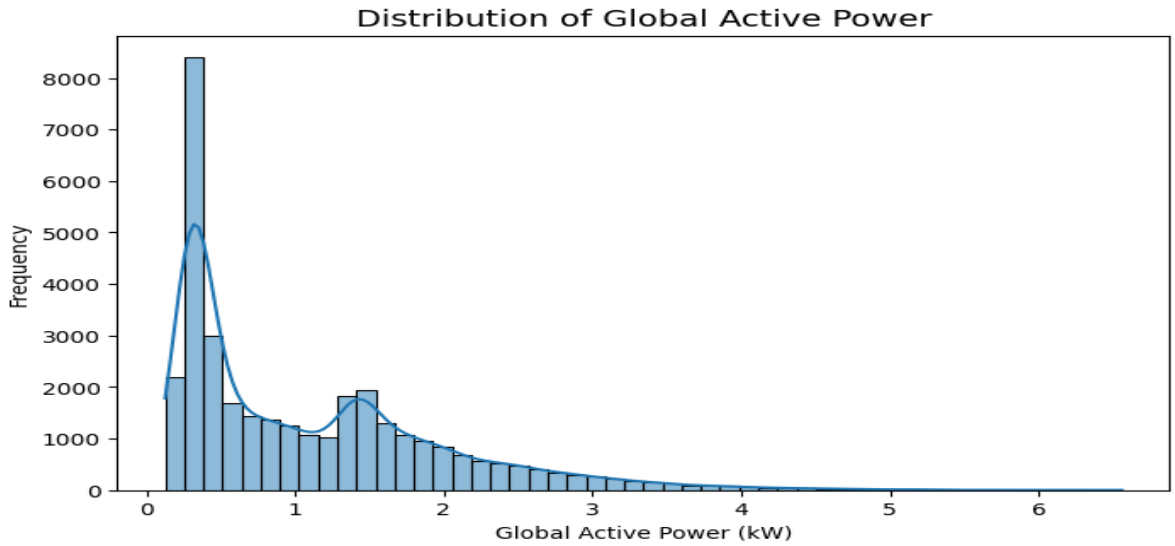


Figure 4.2: Distribution of Global Active Power.

4.1.3 Average Electricity Usage by Hour of Day

Figure 4.3 illustrates the average household electricity consumption by hour of the day, revealing a clear daily usage pattern. Consumption is lowest during the early morning hours, particularly between 1:00 AM and 5:00 AM, and begins to rise from around 6:00 AM as daily activities start. A more pronounced peak occurs in the evening, especially between 8:00 PM and 9:00 PM, when usage reaches its highest levels. This trend reflects typical household behaviour, indicating that the hour of day is a significant factor influencing energy consumption and should be included as an important feature in forecasting models.

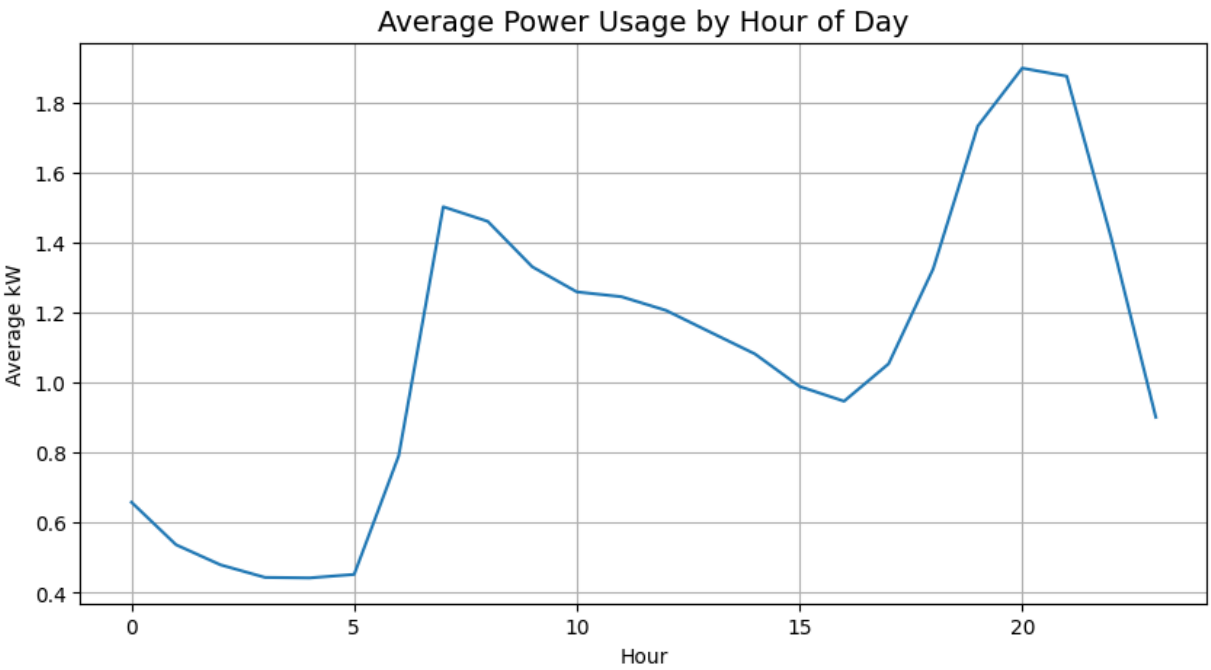


Figure 4.3: Average electricity usage by hour of day.

4.1.4 Average Electricity Usage by Day of Week

Figure 4.4 illustrates the average electricity consumption by day of the week, showing slight variations in usage across different days. Electricity consumption is marginally higher on Saturdays and Sundays compared to weekdays, likely due to increased household occupancy and appliance use during weekends. Although the differences are not substantial, the pattern indicates that weekly behavioural routines influence energy demand. This suggests that the day of the week provides useful temporal information and should be included as a feature to improve forecasting performance.

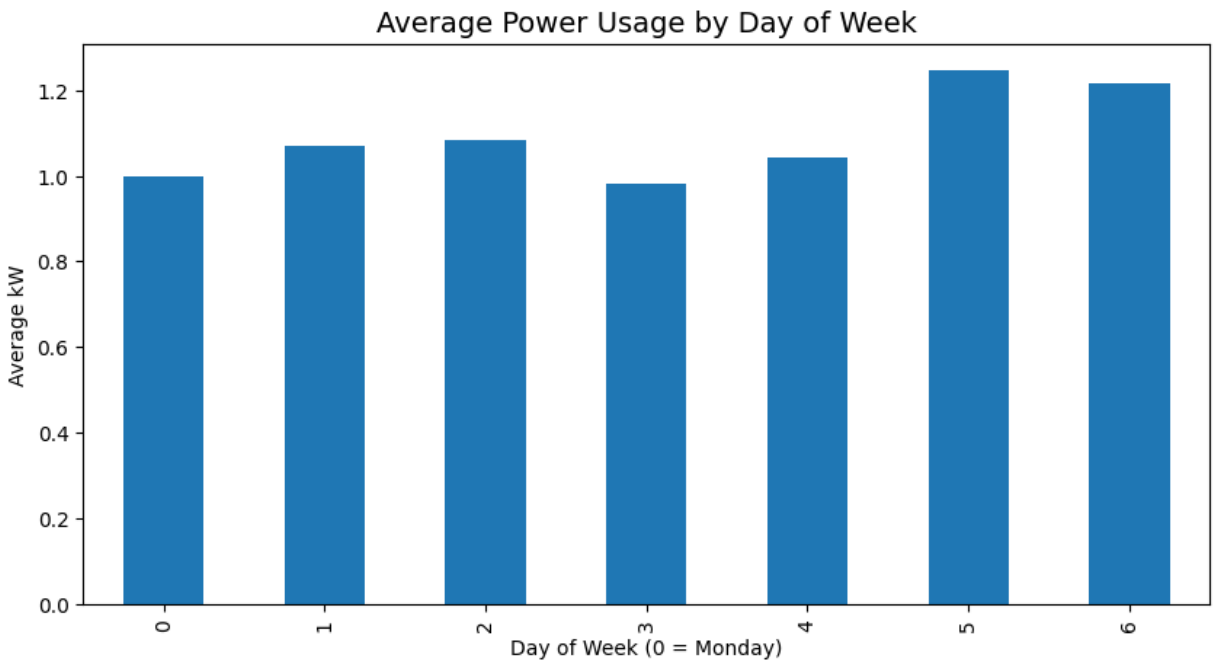


Figure 4.4: Average electricity usage by day of week.

4.1.5 Correlation Analysis After Feature Engineering

Figure 4.5 presents the correlation heatmap of the main numerical and engineered features used in the forecasting models, highlighting key relationships with the target variable, Global Active Power. A very strong positive correlation is observed with Global Intensity, as expected, since both represent related electrical measurements. Other features, such as Sub_metering_3, lag1, and lag24, show moderate to strong positive correlations, indicating their importance in predicting energy consumption. The lag features confirm that past usage is highly informative for forecasting future values, while the 24-hour rolling average also demonstrates a useful relationship by capturing smoothed consumption trends. Overall, the analysis validates the chosen feature-engineering approach and emphasizes the significance of both electrical and temporal features for improving forecasting performance.

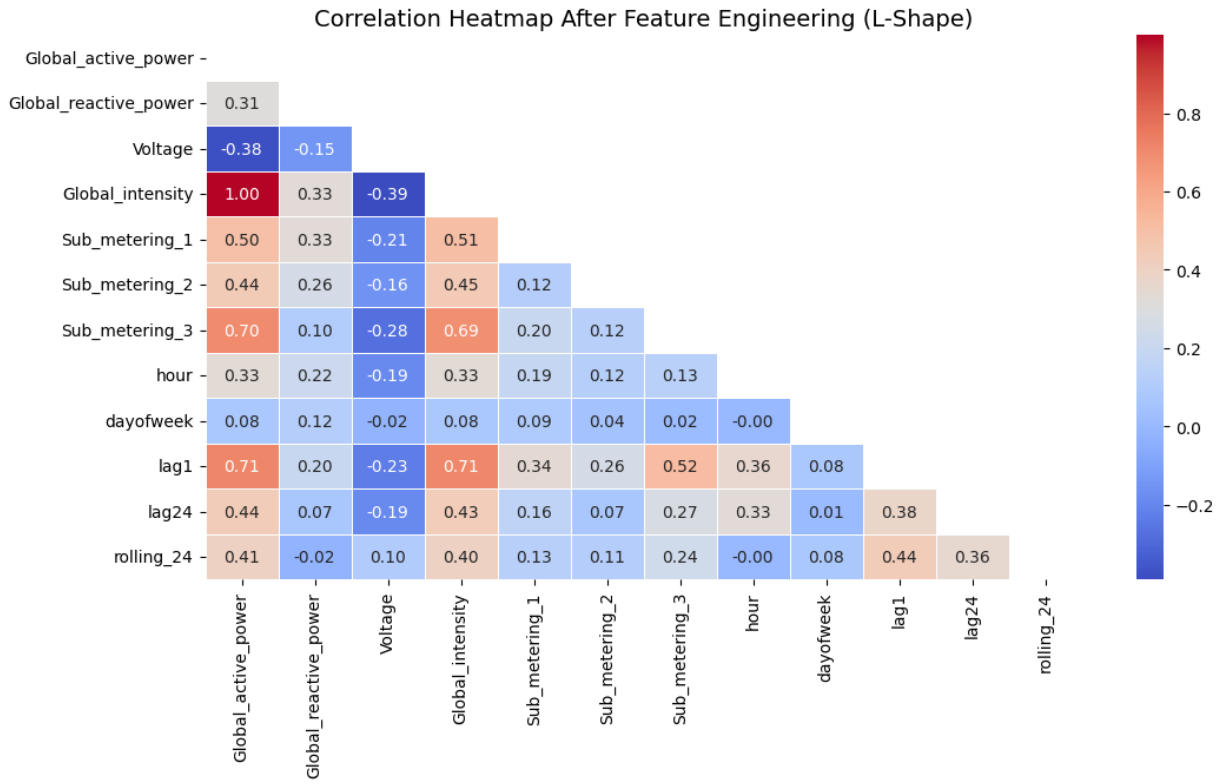


Figure 4.5: Correlation heatmap after feature engineering.

4.2 Forecasting Results

This section presents the performance of the three forecasting models implemented in this study: **RandomForest**, **XGBoost**, and **Prophet**. The models were evaluated using **Root Mean Square Error (RMSE)** and **Mean Absolute Error (MAE)** on the test dataset. In addition to numerical performance, prediction plots and learning curves were used to assess how well each model captured household electricity consumption patterns.

4.2.1 RandomForest Forecasting Performance

The RandomForest model achieved the highest accuracy among all models, with an RMSE of 0.02096 and an MAE of 0.01279. These low error values indicate that the model closely matched actual electricity consumption and effectively captured short-term fluctuations. Its strong performance is attributed to its ability to model nonlinear relationships and complex interactions between lag features, time-based variables, and electrical measurements.

4.2.2 XGBoost Forecasting Performance

The XGBoost model also demonstrated strong predictive capability, achieving an RMSE of 0.03117 and an MAE of 0.01857. While slightly less accurate than RandomForest, it effectively captured variations in energy usage patterns. The marginally higher error may be due to differences in handling feature interactions and short-term volatility. Nevertheless, XGBoost remains a reliable model for energy forecasting tasks.

4.2.3 Prophet Forecasting Performance

The Prophet model showed a different performance profile, with a higher RMSE of 0.62845 and MAE of 0.47906. Although less accurate for short-term predictions, it effectively captured long-

term trends and seasonal patterns, including daily and weekly behaviours. This aligns with its design, which focuses on smooth temporal structures rather than abrupt fluctuations. As a result, Prophet is more suitable for trend analysis and residual-based anomaly detection rather than precise short-term forecasting.

4.2.4 Model Comparison

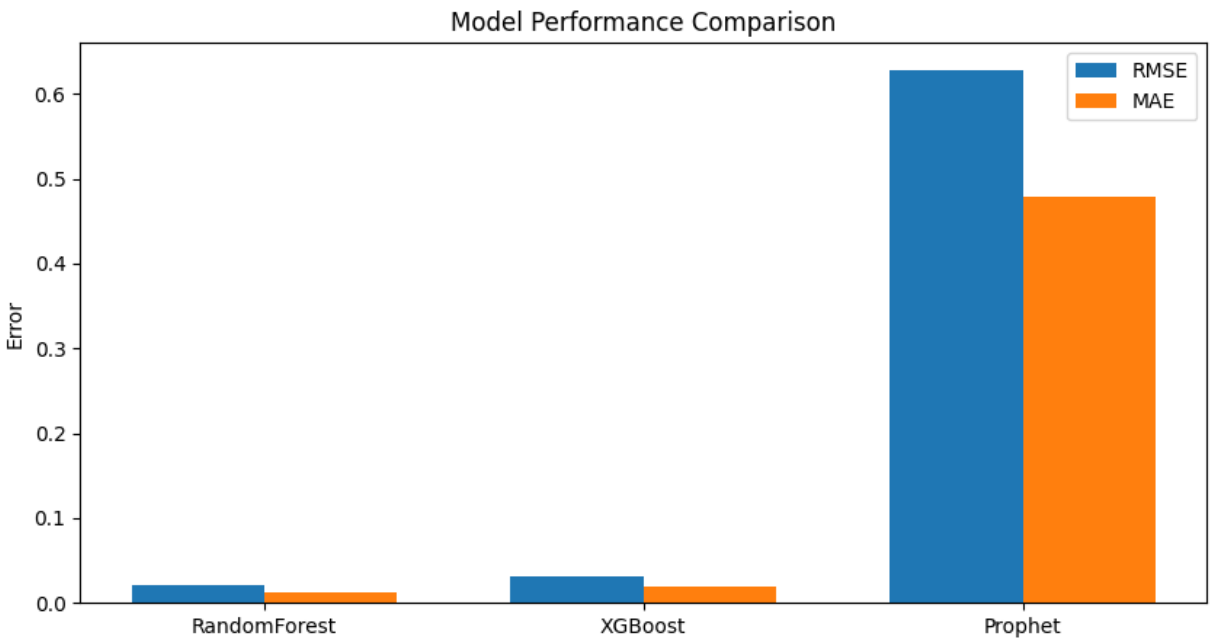


Figure 4.5.1: summarises the forecasting performance of all three models.

4.2.5 Visual Analysis of Forecasting Models

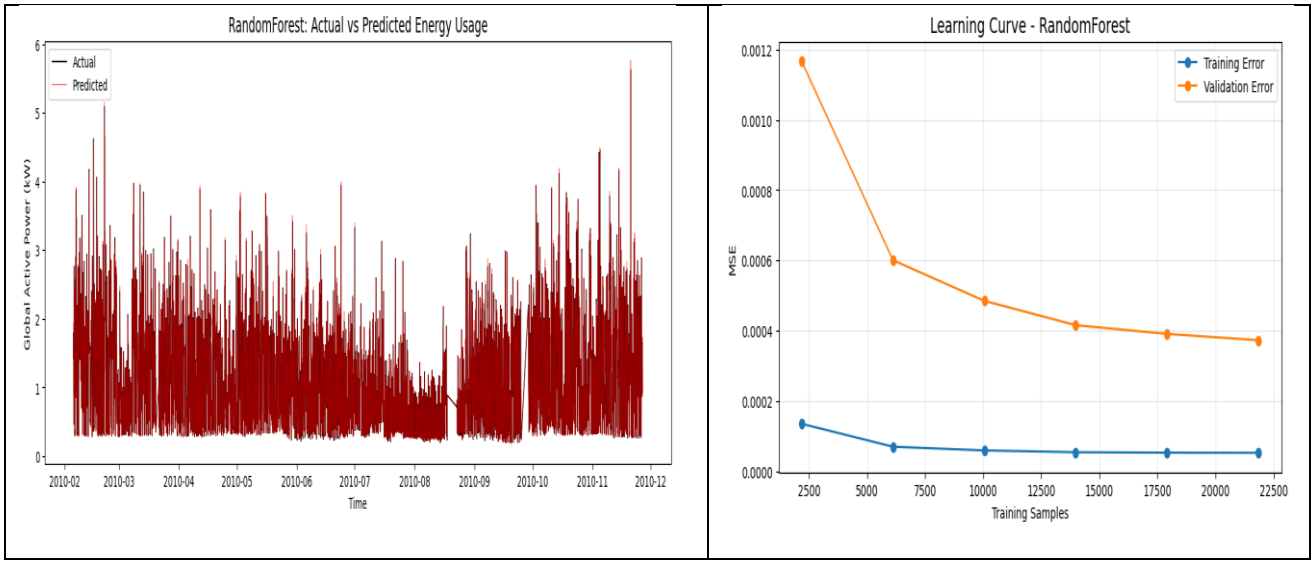
In addition to numerical evaluation using RMSE and MAE, the forecasting models were also assessed visually through prediction plots, learning curves, feature importance analysis, and seasonal decomposition. These visual outputs provide further insight into how well each model learned the structure of the data and which patterns it captured most effectively.

4.2.5.1 Model Forecasting Performance and Analysis

This section evaluates the forecasting behaviour of RandomForest, XGBoost, and Prophet using prediction plots, learning curves, and feature importance.

RandomForest:

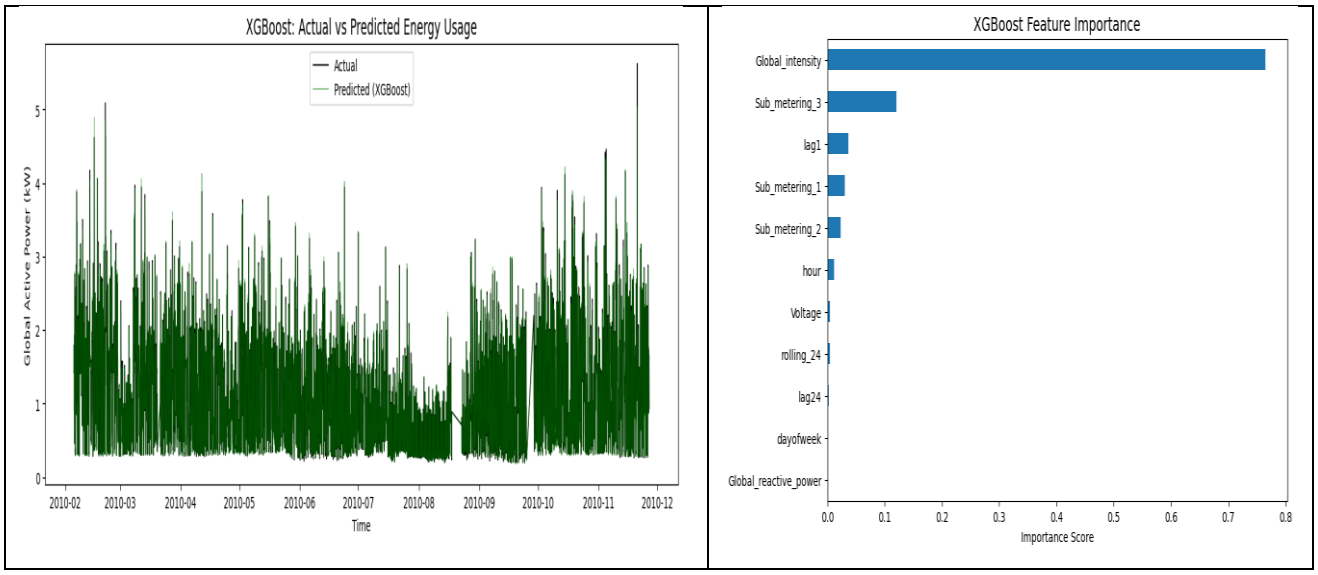
Figures 4.6 and 4.7 show that predicted values closely match actual consumption, capturing both fluctuations and peak usage effectively. The learning curve indicates low training error and decreasing validation error, suggesting good generalisation with minimal overfitting.

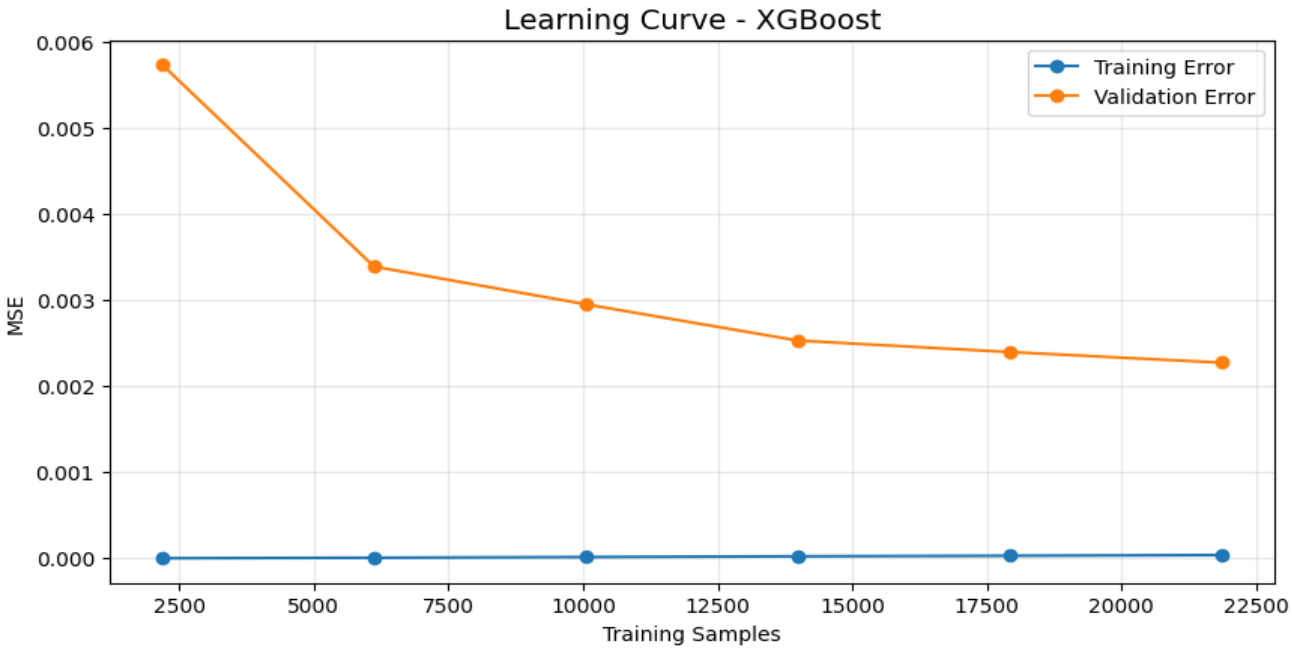


Figures 4.6 and 4.7: RandomForest Model Performance and Learning Curve

XGBoost:

Figures 4.8,4.9, and 4.10 demonstrate that XGBoost also follows the overall consumption pattern, though with slight deviations during peak periods, consistent with its higher error values. Feature importance highlights Global Intensity, Sub_metering_3, and lag features as key predictors. The learning curve shows effective learning, though with slightly higher validation error than RandomForest.





Figures 4.8, 4.9, and 4.10: XGBoost Model: Prediction Performance and Learning Curve

Prophet:

Figures 4.11 and 4.12 show that Prophet produces smoother forecasts, capturing trend and seasonality but not short-term spikes. The components reveal clear daily and weekly patterns, consistent with earlier analysis. Despite higher error values, Prophet is useful for modelling long-term behaviour and temporal structure.

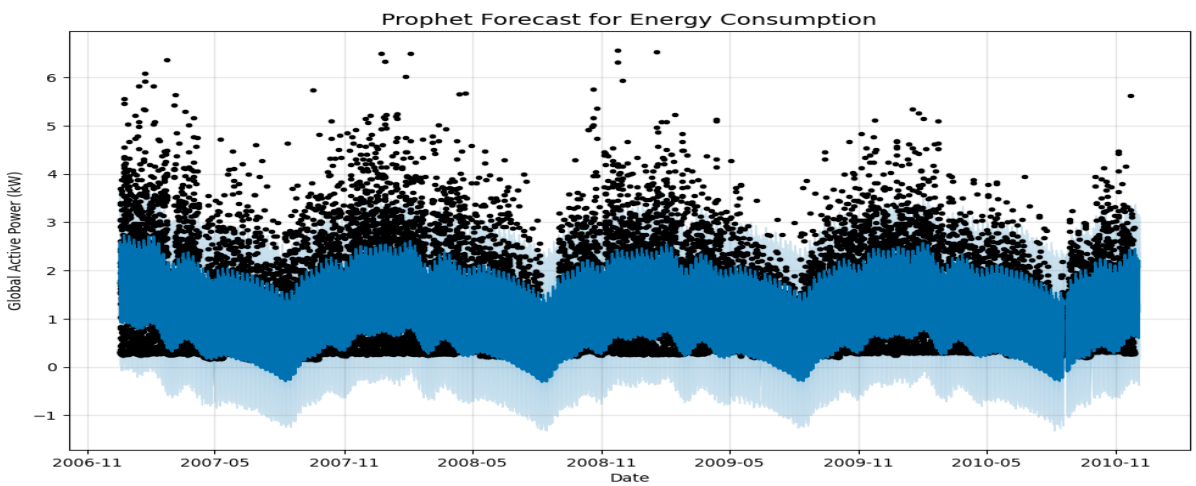


Figure 4.11: Prophet forecast for household electricity consumption.

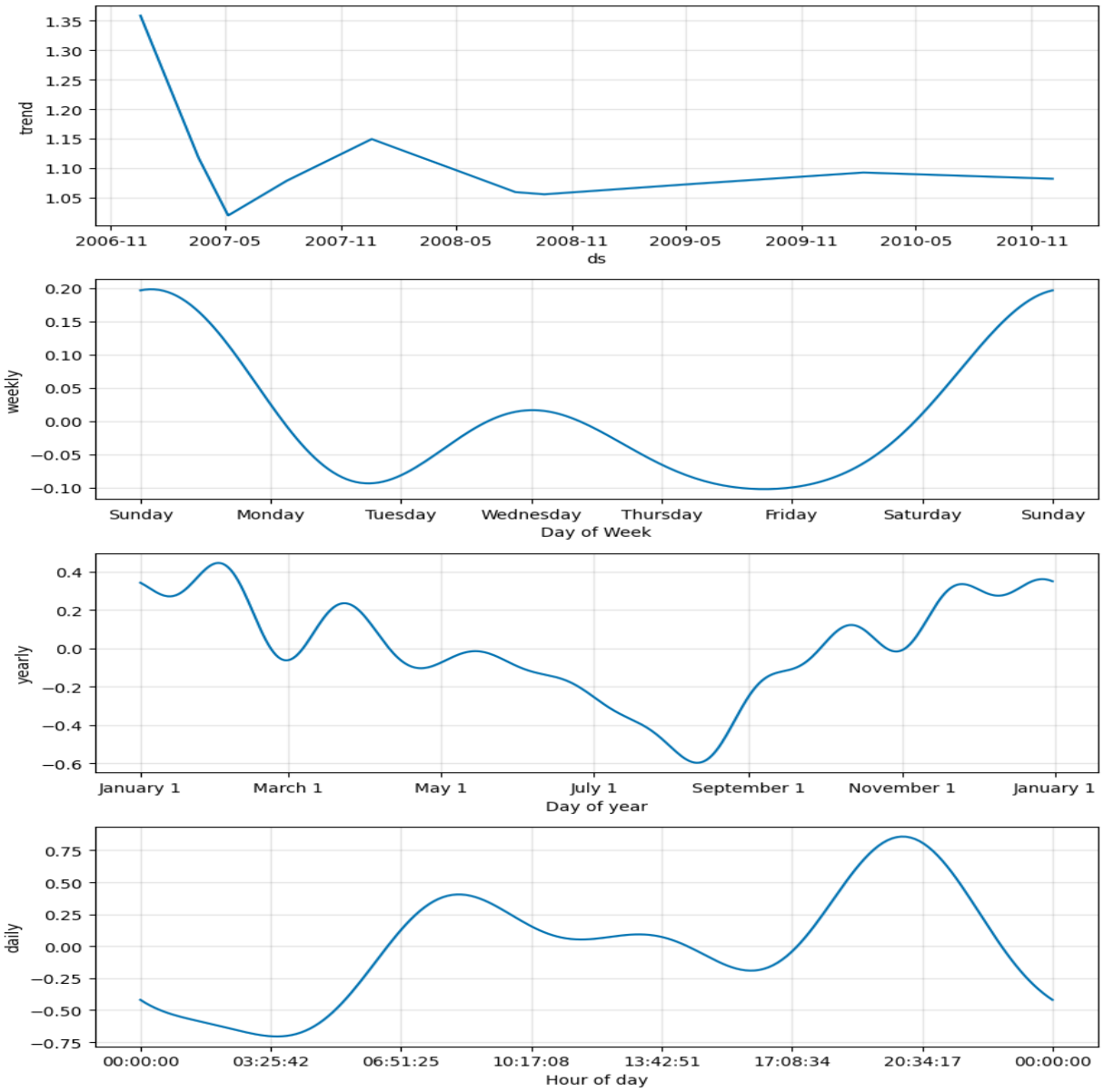


Figure 4.12: Prophet trend and seasonal components.

4.3 Anomaly Detection Results

In addition to forecasting future electricity consumption, this study also aimed to identify abnormal or unusual usage patterns. Three anomaly detection techniques were applied:

1. **Rolling Z-Score**
2. **Isolation Forest**
3. **Prophet Residual Analysis**

Each method was used to identify data points that deviated significantly from normal household electricity consumption behaviour.

Rolling Z-Score:

This method detected **273 anomalies** using a threshold of |Z| > 3 based on rolling mean and standard deviation. It effectively identifies extreme spikes but is limited to univariate statistical deviations and may miss more complex patterns.

Isolation Forest:

Isolation Forest identified **342 anomalies**, capturing more irregular behaviour than the Z-score method. By using multiple features, it detects both extreme values and unusual combinations, making it more flexible and data-driven.

Prophet Residuals:

This approach detected **413 anomalies**, the highest among all methods. It identifies deviations from expected trends and seasonality, capturing a wider range of anomalies. However, due to the Prophet's smoother forecasts, it may over-detect anomalies, especially for short-term fluctuations.

4.3.1 Comparison of Anomaly Detection Methods

Method	**Number of Anomalies**
Rolling Z-Score	273
Isolation Forest	342
Prophet Residual Analysis	413

Table 4.2 summarises the number of anomalies detected by each method.

The results show that different anomaly detection methods identify different numbers of abnormal events, reflecting their different detection strategies. The rolling Z-score method was the most conservative, Isolation Forest provided a more flexible multivariate perspective, and Prophet residual analysis was the most sensitive to deviations from expected time-series behaviour.

These findings demonstrate that anomaly detection in household electricity consumption is not a one-method problem, and that different techniques can reveal different aspects of unusual behaviour.

4.3.2 Visual Analysis of Anomaly Detection Results

In addition to the numerical anomaly counts, the anomaly detection methods were analysed visually to understand how each technique identified unusual electricity consumption behaviour over time.

4.3.2.1 Rolling Z-Score Anomalies

Figure 4.13 shows the anomalies detected using the rolling Z-score method, highlighted as red points on the electricity consumption time series. The anomalies are mainly concentrated around high-consumption spikes, indicating strong deviations from the local average. This confirms that the method is effective for detecting extreme changes, but its reliance on statistical thresholds limits its ability to capture more subtle or complex anomalies.

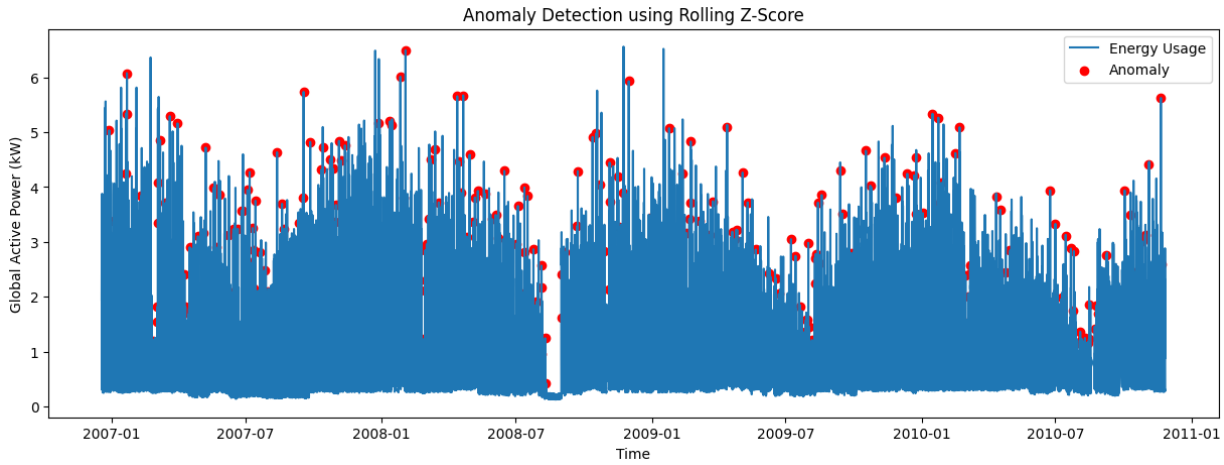


Figure 4.13: Anomalies detected using the rolling Z-score method.

4.3.2.2 Isolation Forest Anomalies

Figure 4.14 shows the anomalies detected by the Isolation Forest model, highlighting more anomalous points than the rolling Z-score method, including spikes spread across the entire time period. This indicates higher sensitivity to unusual consumption patterns, capturing not only extreme values but also contextually abnormal behaviour. By considering multiple features, Isolation Forest provides a more flexible and comprehensive approach than threshold-based methods.

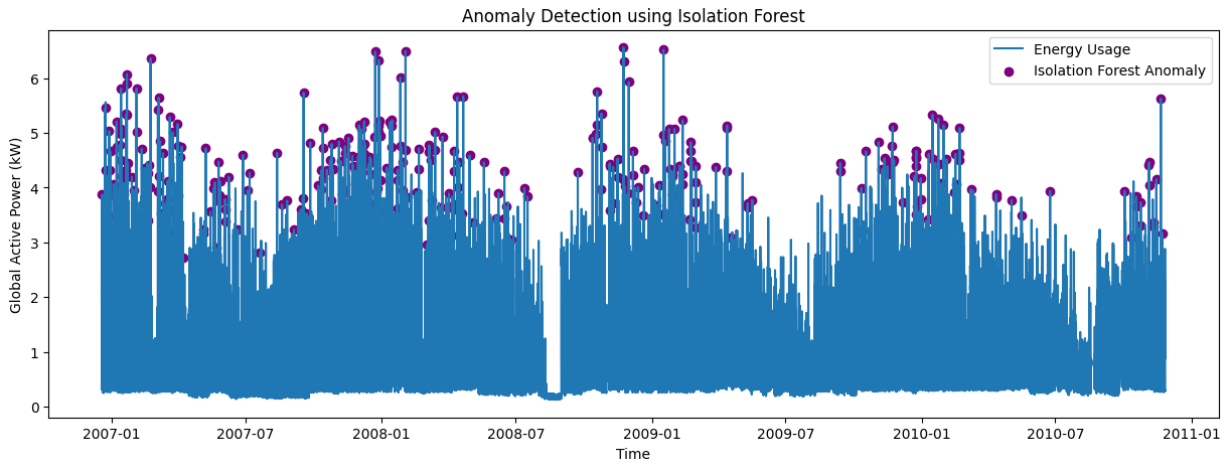


Figure 4.14: Anomalies detected using Isolation Forest.

4.3.2.3 Prophet Residual Anomalies

Figure 4.15 presents the anomalies detected using Prophet residual analysis, based on deviations between actual and predicted consumption values. The method identifies a large number of anomalies, particularly around sharp spikes and irregular changes, making it the most sensitive among the three approaches. Since Prophet captures overall trend and seasonality rather than short-term fluctuations, sudden deviations result in large residuals, making this approach effective for detecting departures from expected temporal patterns.

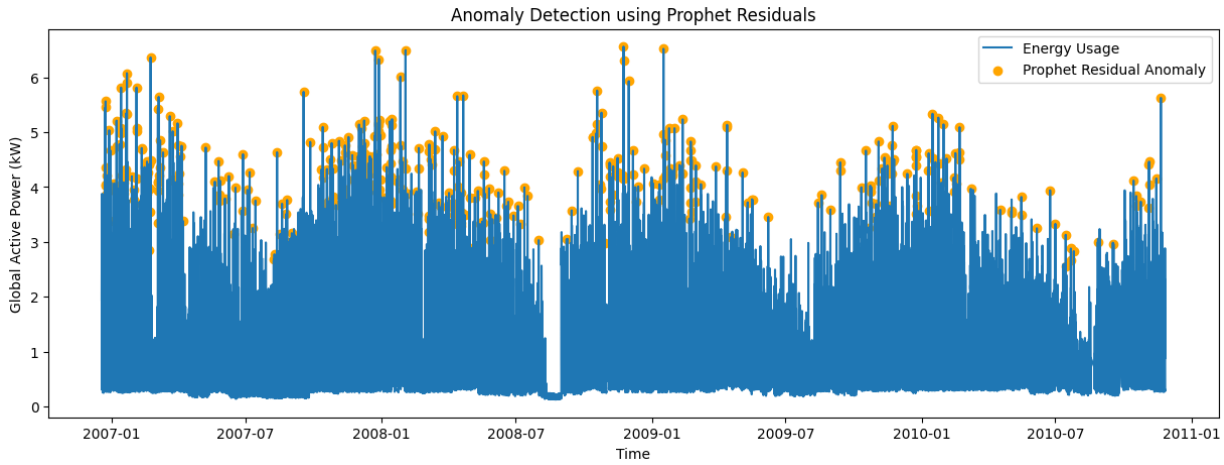


Figure 4.15: Anomalies detected using Prophet residual analysis.

4.3.2.4 Comparison of Anomaly Detection Methods

Figure 4.16 provides a direct comparison of the number of anomalies detected by each method. The bar chart clearly shows that:

- **Rolling Z-score** detected the fewest anomalies (**273**)
- **Isolation Forest** detected a moderate number (**342**)
- **Prophet residual analysis** detected the highest number (**413**)

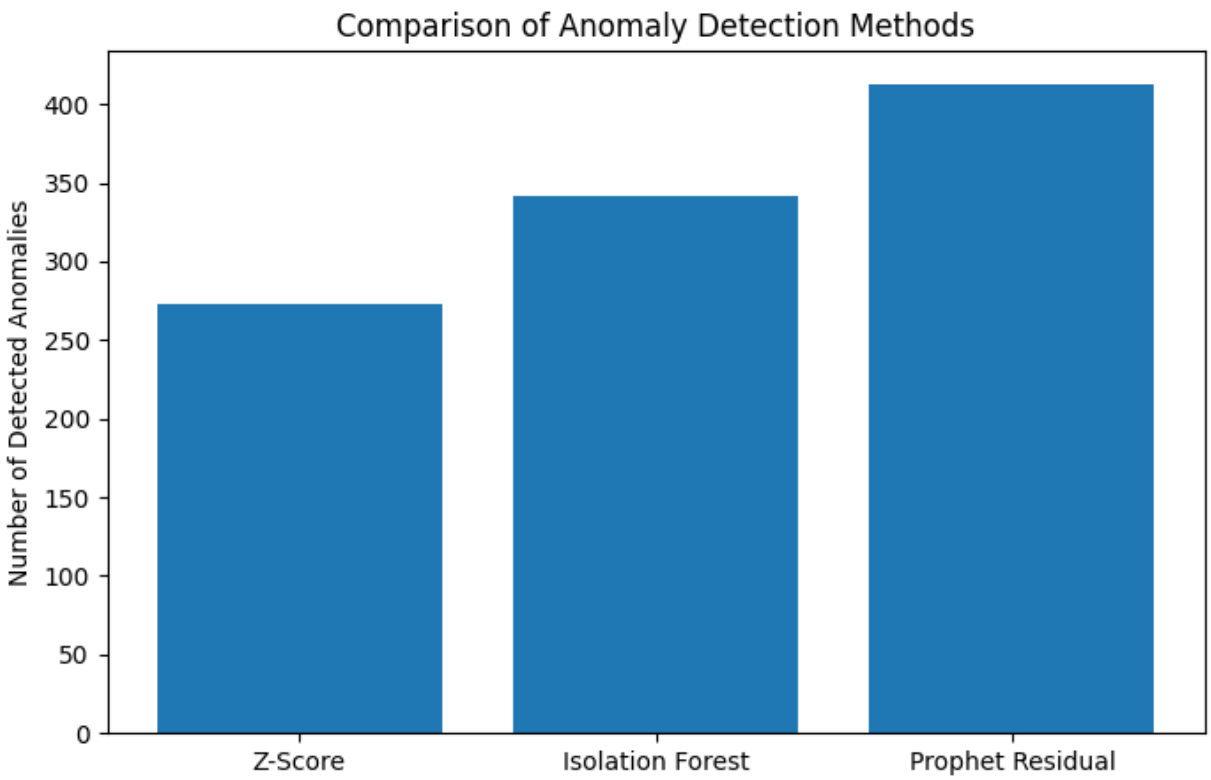


Figure 4.16: Comparison of anomaly detection methods.

4.4 Discussion of Findings

The findings of this study demonstrate that machine learning models are highly effective for forecasting household electricity consumption when combined with appropriate preprocessing and feature engineering. RandomForest achieved the best performance, followed by XGBoost, while Prophet showed higher prediction errors due to its focus on trend and seasonality rather than short-term fluctuations.

The strong performance of RandomForest and XGBoost is attributed to their ability to model nonlinear relationships and effectively utilise engineered features such as lag variables, rolling averages, and electrical measurements like Global Intensity. This is further supported by the prediction plots, where both models closely follow actual consumption patterns, while Prophet produces smoother forecasts that miss sharp peaks.

In terms of anomaly detection, Rolling Z-score identified extreme spikes, Isolation Forest captured more flexible and multivariate anomalies, and Prophet residual analysis was the most sensitive to deviations from expected behaviour. These differences highlight that each method provides complementary insights into abnormal energy usage.

Overall, the study confirms that combining forecasting and anomaly detection within a unified framework is effective and valuable for smart energy systems, enabling accurate prediction and early identification of unusual consumption patterns.

Chapter 5 – Conclusion and Future Work

5.1 Summary of the Study

This study aimed to develop and evaluate time-series machine learning models for forecasting household electricity consumption and detecting anomalous usage patterns. To achieve this, the Individual Household Electric Power Consumption dataset was used, and a structured methodology was followed, including data preprocessing, feature engineering, model development, and performance evaluation.

Exploratory Data Analysis revealed that household electricity consumption exhibits strong temporal patterns, including daily and weekly trends, as well as significant variability and occasional extreme spikes. Based on these findings, relevant features such as hour of day, day of week, lag variables, and rolling averages were engineered to improve model performance.

Three forecasting models, RandomForest, XGBoost, and Prophet, were implemented and evaluated using RMSE and MAE. In addition, three anomaly detection methods, rolling Z-score, Isolation Forest, and Prophet residual analysis, were applied to identify abnormal energy usage patterns.

5.2 Key Findings

The results of this study demonstrate that machine learning models are highly effective for forecasting household electricity consumption when appropriate feature engineering techniques are applied.

- **RandomForest achieved the best performance**, with the lowest RMSE and MAE values, indicating high accuracy in predicting short-term electricity consumption.
- **XGBoost also performed strongly**, producing accurate predictions but with slightly higher error values compared to RandomForest.
- **Prophet was effective in modelling trend and seasonality**, but it was less accurate for short-term forecasting due to its inability to capture rapid fluctuations and sharp spikes in the data.

The anomaly detection analysis revealed that:

- **Rolling Z-score** effectively identified extreme spikes in electricity consumption.
- **Isolation Forest** detected a wider range of anomalies by considering multiple features.
- **Prophet residual analysis** identified the largest number of anomalies, highlighting deviations from expected temporal patterns.

These findings confirm that combining forecasting and anomaly detection provides a more comprehensive understanding of household electricity consumption behaviour.

5.3 Research Contributions

This study makes several important contributions:

- It provides a **comparative analysis of machine learning and time-series models** for household electricity forecasting.

- It demonstrates the effectiveness of **feature engineering techniques**, including lag features and rolling statistics, in improving prediction accuracy.
- It integrates **forecasting and anomaly detection within a single framework**, which is less commonly addressed in existing studies.
- It highlights the strengths and limitations of different anomaly detection methods when applied to real-world energy consumption data.

These contributions are relevant for the development of smart energy systems and intelligent energy management solutions.

5.4 Limitations of the Study

Despite the promising results, this study has several limitations.

First, the dataset used represents electricity consumption from a **single household**, which may limit the generalisability of the findings to other households or regions. Energy usage patterns can vary significantly depending on household characteristics, geographical location, and lifestyle factors.

Second, the study focused on a limited set of models—RandomForest, XGBoost, and Prophet. Although these models provided strong results, more advanced deep learning approaches such as Long Short-Term Memory (LSTM) networks or Transformer-based models were not explored due to time and computational constraints.

Third, hyperparameter tuning was not extensively performed for all models. While the models achieved good performance with default or basic settings, further optimisation could potentially improve prediction accuracy.

Finally, anomaly detection results were evaluated primarily through visual inspection and anomaly counts rather than using labelled ground truth data. This makes it difficult to quantitatively assess the accuracy of anomaly detection methods.

5.5 Future Work

Several directions for future research can be identified based on the limitations of this study.

- Future work could involve applying the proposed methodology to **multiple households or larger datasets** to improve generalisability.
- Advanced models such as **LSTM, GRU, or Transformer-based architectures** could be implemented to capture long-term dependencies and improve forecasting performance.
- More comprehensive **hyperparameter tuning techniques**, such as Grid Search or Bayesian Optimisation, could be applied to further optimise model performance.
- Additional features, including **weather data, occupancy information, or appliance-level data**, could be incorporated to enhance prediction accuracy.
- Future studies could also explore **real-time anomaly detection systems** for practical deployment in smart homes or smart grid environments.

- Finally, the use of **labelled anomaly datasets** would allow for more rigorous evaluation of anomaly detection performance.

5.6 Conclusion

In conclusion, this study successfully demonstrated that time-series machine learning models can be effectively used to forecast household electricity consumption and detect anomalous usage patterns. The results showed that RandomForest and XGBoost outperform Prophet in short-term prediction accuracy, while Prophet provides valuable insights into trend and seasonal behaviour. The integration of multiple anomaly detection methods further enhanced the analysis by identifying different types of abnormal consumption events.

Overall, the study highlights the potential of combining forecasting and anomaly detection techniques to support smarter and more efficient energy management systems. The findings contribute to the growing body of research in energy analytics and provide a foundation for future developments in intelligent energy forecasting and monitoring solutions.

References

Adhikari, R. and Agrawal, R. (2013) 'An introductory study on time series modeling and forecasting', *arXiv preprint arXiv:1302.6613*.

Ahmed, H.A. and Khamees, M. (2025) 'Energy consumption forecasting using naïve method, ARIMA, and auto-ARIMA', *Mesopotamian Journal of Big Data*, 2025(1), pp. 36–41.

Cascone, L., Dell'Anna, F., Mokhtari, G. and De Masi, R.F. (2023) 'Multi-step household electricity forecasting in smart and sustainable homes using hybrid ConvLSTM model', *Energy and Buildings*, 292, p. 113190.

Dua, D. and Graff, C. (2019) *UCI Machine Learning Repository*. Irvine, CA: University of California, School of Information and Computer Science. Available at: http://archive.ics.uci.edu/ml (Accessed: 7 April 2026).

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M. and Duchesnay, E. (2011) 'Scikit-learn: Machine learning in Python', *Journal of Machine Learning Research*, 12, pp. 2825–2830.

Syed, D., Zainab, A., Ghrayeb, A., Refaat, S.S., Abu-Rub, H. and Bouhali, O. (2021) 'Smart grid big data analytics: Survey of technologies, techniques, and applications', *IEEE Access*, 9, pp. 59564–59585.

Taylor, S.J. and Letham, B. (2018) 'Forecasting at scale', *The American Statistician*, 72(1), pp. 37–45.

Chen, T. and Guestrin, C. (2016) 'XGBoost: A scalable tree boosting system', in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. New York: ACM, pp. 785–794.

Breiman, L. (2001) 'Random forests', *Machine Learning*, 45(1), pp. 5–32.

Liu, F.T., Ting, K.M. and Zhou, Z.-H. (2008) 'Isolation forest', in *2008 Eighth IEEE International Conference on Data Mining*. Pisa: IEEE, pp. 413–422.

Appendix

```
#  Import Required Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, learning_curve
from sklearn.metrics import mean_absolute_error, mean_squared_error
from sklearn.ensemble import RandomForestRegressor
from xgboost import XGBRegressor
from prophet import Prophet
import warnings
warnings.filterwarnings("ignore")
# -------------------------------
# Load Dataset
# -------------------------------
df = pd.read_csv("/content/sample_data/household_power_consumption.txt",
            sep=';',
            parse_dates={'DateTime':['Date','Time']},
            infer_datetime_format=True,
            low_memory=False,
            na_values=['?'])
df.head()
df.shape

# -------------------------------
# Data Cleaning
# -------------------------------
# Convert numeric columns
numeric_cols = ['Global_active_power','Global_reactive_power','Voltage',
```

```
            'Global_intensity','Sub_metering_1','Sub_metering_2','Sub_metering_3']
df[numeric_cols] = df[numeric_cols].astype(float)
# Drop missing values
df.dropna(inplace=True)
# Set datetime index
df.set_index('DateTime', inplace=True)
df.head()
# -------------------------------
# Resample to hourly data
# -------------------------------
df_hourly = df.resample('H').mean()
df_hourly = df_hourly.dropna()
df_hourly.head()
# -------------------------------
#  Feature Engineering
# -------------------------------
df_hourly['hour'] = df_hourly.index.hour
df_hourly['dayofweek'] = df_hourly.index.dayofweek

# Lag features
df_hourly['lag1'] = df_hourly['Global_active_power'].shift(1)
df_hourly['lag24'] = df_hourly['Global_active_power'].shift(24)
# Rolling mean
df_hourly['rolling_24'] = df_hourly['Global_active_power'].rolling(window=24).mean()
df_hourly.dropna(inplace=True)
df_hourly.head()
# -----------------------------------------------------
# Time-Series Plot of Global Active Power
# -----------------------------------------------------
plt.figure(figsize=(15,4))
```

```
plt.plot(df_hourly['Global_active_power'], color='blue')
plt.title("Global Active Power (Hourly) - First Look", fontsize=14)
plt.xlabel("Time")
plt.ylabel("kW")
plt.show()
# -------------------------------------------------------
# Distribution of Global Active Power
# -------------------------------------------------------
plt.figure(figsize=(8,5))
sns.histplot(df_hourly['Global_active_power'], kde=True, bins=50)
plt.title("Distribution of Global Active Power", fontsize=14)
plt.xlabel("Global Active Power (kW)")
plt.ylabel("Frequency")
plt.show()
# -------------------------------------------------------
# Average Usage by Hour
# -------------------------------------------------------
plt.figure(figsize=(10,5))
df_hourly.groupby(df_hourly.index.hour)['Global_active_power'].mean().plot()
plt.title("Average Power Usage by Hour of Day", fontsize=14)
plt.xlabel("Hour")
plt.ylabel("Average kW")
plt.grid(True)
plt.show()
# -------------------------------------------------------
# Average Usage by Day of Week
# -------------------------------------------------------
plt.figure(figsize=(10,5))
df_hourly.groupby(df_hourly.index.dayofweek)['Global_active_power'].mean().plot(kind='bar'
)
```

```
plt.title("Average Power Usage by Day of Week", fontsize=14)
plt.xlabel("Day of Week (0 = Monday)")
plt.ylabel("Average kW")
plt.show()
# -------------------------------------------------------
# L-Shaped Correlation Heatmap
# -------------------------------------------------------
plt.figure(figsize=(12,6))
corr_cols = ['Global_active_power','Global_reactive_power','Voltage',
            'Global_intensity','Sub_metering_1','Sub_metering_2','Sub_metering_3',
            'hour','dayofweek','lag1','lag24','rolling_24']
corr = df_hourly[corr_cols].corr()

# Create mask for L-shape (upper triangle mask)
mask = np.triu(np.ones_like(corr, dtype=bool))
sns.heatmap(
    corr,
    mask=mask,                  # L-shape mask
    annot=True,                 # numbers inside cells
    cmap='coolwarm',
    fmt=".2f",
    linewidths=0.5
)
plt.title("Correlation Heatmap After Feature Engineering (L-Shape)", fontsize=14)
plt.show()
# -------------------------------------------------------
#  Time-Series Train/Test Split (No Shuffle)
# -------------------------------------------------------
# Define target and features
target = 'Global_active_power'
```

```
feature_cols = ['Global_reactive_power','Voltage','Global_intensity',
            'Sub_metering_1','Sub_metering_2','Sub_metering_3',
            'hour','dayofweek','lag1','lag24','rolling_24']
X = df_hourly[feature_cols]
y = df_hourly[target]
# Time-series split: 80% train, 20% test
split_point = int(len(df_hourly) * 0.8)
X_train = X.iloc[:split_point]
X_test  = X.iloc[split_point:]

y_train = y.iloc[:split_point]
y_test  = y.iloc[split_point:]
print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))
print("Train start:", X_train.index.min(), "→ Train end:", X_train.index.max())
print("Test start:", X_test.index.min(),   "→ Test end:", X_test.index.max())
X_train.shape, X_test.shape, y_train.shape, y_test.shape
# -------------------------------------------------------
# Train RandomForest Model
# -------------------------------------------------------
rf_model = RandomForestRegressor(
   n_estimators=200,
   max_depth=15,
   random_state=42,
   n_jobs=-1
)
rf_model.fit(X_train, y_train)
print("RandomForest model trained successfully!")
# -------------------------------------------------------
# Evaluation Metrics (RMSE, MAE)
```

```
# -------------------------------------------------------
rf_preds = rf_model.predict(X_test)
rmse = np.sqrt(mean_squared_error(y_test, rf_preds))
mae = mean_absolute_error(y_test, rf_preds)

print("RandomForest Performance:")
print("--------------------------")
print("RMSE:", rmse)
print("MAE :", mae)
# -------------------------------------------------------
# Prediction vs Actual Plot
# -------------------------------------------------------
plt.figure(figsize=(15,5))
plt.plot(y_test.index, y_test, label='Actual', color='black', linewidth=1)
plt.plot(y_test.index, rf_preds, label='Predicted', color='red', linewidth=0.8, alpha=0.6)
plt.title("RandomForest: Actual vs Predicted Energy Usage", fontsize=14)
plt.ylabel("Global Active Power (kW)")
plt.xlabel("Time")
plt.legend()
plt.show()
# -------------------------------------------------------
# Learning Curve for RandomForest
# -------------------------------------------------------
train_sizes, train_scores, test_scores = learning_curve(
    estimator=rf_model,
    X=X_train,
    y=y_train,
    cv=5,
    scoring='neg_mean_squared_error',
    n_jobs=-1,
```

```
    train_sizes=np.linspace(0.1, 1.0, 6)
)
train_mean = -train_scores.mean(axis=1)
test_mean = -test_scores.mean(axis=1)
plt.figure(figsize=(10,5))
plt.plot(train_sizes, train_mean, 'o-', label="Training Error")
plt.plot(train_sizes, test_mean, 'o-', label="Validation Error")
plt.grid(True, alpha=0.3)
plt.title("Learning Curve - RandomForest", fontsize=14)
plt.xlabel("Training Samples")
plt.ylabel("MSE")
plt.legend()
plt.show()
# -------------------------------------------------------
# CELL 16: Train XGBoost Model
# -------------------------------------------------------
xgb_model = XGBRegressor(
    n_estimators=300,
    max_depth=10,
    learning_rate=0.05,
    subsample=0.8,
    colsample_bytree=0.8,
    random_state=42,
    n_jobs=-1
)
xgb_model.fit(X_train, y_train)
print("XGBoost model trained successfully!")
# -------------------------------------------------------
# XGBoost Evaluation (RMSE, MAE)
# -------------------------------------------------------
```

```
xgb_preds = xgb_model.predict(X_test)

rmse_xgb = np.sqrt(mean_squared_error(y_test, xgb_preds))

mae_xgb = mean_absolute_error(y_test, xgb_preds)

print("XGBoost Performance:")

print("----------------------")

print("RMSE:", rmse_xgb)

print("MAE :", mae_xgb)

# -------------------------------------------------------

# XGBoost Predictions vs Actual

# -------------------------------------------------------

plt.figure(figsize=(15,5))

plt.plot(y_test.index, y_test, label='Actual', color='black', linewidth=1)

plt.plot(y_test.index, xgb_preds, label='Predicted (XGBoost)', color='green', linewidth=0.8, alpha=0.6)

plt.title("XGBoost: Actual vs Predicted Energy Usage", fontsize=14)

plt.ylabel("Global Active Power (kW)")

plt.xlabel("Time")

plt.legend()

plt.show()

# -------------------------------------------------------

# XGBoost Feature Importance

# -------------------------------------------------------

importances = pd.Series(xgb_model.feature_importances_, index=X_train.columns)

importances = importances.sort_values()

plt.figure(figsize=(10,6))

importances.plot(kind='barh')

plt.title("XGBoost Feature Importance", fontsize=14)

plt.xlabel("Importance Score")

plt.show()

# -------------------------------------------------------
```

```
# Learning Curve - XGBoost
# -----------------------------------------------------
train_sizes, train_scores, test_scores = learning_curve(
    estimator=xgb_model,
    X=X_train,
    y=y_train,
    cv=5,
    scoring='neg_mean_squared_error',
    n_jobs=-1,
    train_sizes=np.linspace(0.1, 1.0, 6)
)
train_mean = -train_scores.mean(axis=1)
test_mean = -test_scores.mean(axis=1)
plt.figure(figsize=(10,5))
plt.plot(train_sizes, train_mean, 'o-', label="Training Error")
plt.plot(train_sizes, test_mean, 'o-', label="Validation Error")
plt.grid(True, alpha=0.3)
plt.title("Learning Curve - XGBoost", fontsize=14)
plt.xlabel("Training Samples")
plt.ylabel("MSE")
plt.legend()
plt.show()
# -----------------------------------------------------
#  Prepare Data for Prophet
# -----------------------------------------------------
prophet_df = df_hourly.reset_index()[['DateTime','Global_active_power']]
prophet_df.columns = ['ds','y']
prophet_df.head()
# -----------------------------------------------------
#  Train Prophet Model
```

```
# -------------------------------------------------------
prophet_model = Prophet(
    daily_seasonality=True,
    weekly_seasonality=True,
    yearly_seasonality=True
)
prophet_model.fit(prophet_df)
print("Prophet model trained successfully!")
# -------------------------------------------------------
# Create Future Dataframe
# -------------------------------------------------------
future = prophet_model.make_future_dataframe(periods=168, freq='H')
future.tail()
# -------------------------------------------------------
# Prophet Forecast
# -------------------------------------------------------
forecast = prophet_model.predict(future)
forecast[['ds','yhat','yhat_lower','yhat_upper']].tail()
# -------------------------------------------------------
# Prophet Forecast Plot
# -------------------------------------------------------
fig = prophet_model.plot(forecast)
plt.title("Prophet Forecast for Energy Consumption", fontsize=14)
plt.xlabel("Date")
plt.ylabel("Global Active Power (kW)")
plt.show()
# -------------------------------------------------------
# Prophet Components
# -------------------------------------------------------
prophet_model.plot_components(forecast)
```

```
plt.show()
# -----------------------------------------------------
# Prophet Evaluation
# -----------------------------------------------------
prophet_test = forecast.set_index('ds').loc[y_test.index]
rmse_prophet = np.sqrt(mean_squared_error(y_test, prophet_test['yhat']))
mae_prophet = mean_absolute_error(y_test, prophet_test['yhat'])
print("Prophet Performance:")
print("--------------------")
print("RMSE:", rmse_prophet)
print("MAE :", mae_prophet)
# -----------------------------------------------------
# Model Performance Comparison
# -----------------------------------------------------
models = ['RandomForest','XGBoost','Prophet']
rmse_values = [0.02096, 0.03117, 0.62845]
mae_values  = [0.01279, 0.01857, 0.47906]
comparison_df = pd.DataFrame({
    'Model': models,
    'RMSE': rmse_values,
    'MAE': mae_values
})
comparison_df
plt.figure(figsize=(10,5))
x = np.arange(len(models))
width = 0.35
plt.bar(x - width/2, rmse_values, width, label='RMSE')
plt.bar(x + width/2, mae_values, width, label='MAE')
plt.xticks(x, models)
plt.title("Model Performance Comparison")
```

```
plt.ylabel("Error")
plt.legend()
plt.show()

# -------------------------------------------------------
# Rolling Statistics for Anomaly Detection
# -------------------------------------------------------
window = 24   # 24 hours window
df_hourly['rolling_mean'] = df_hourly['Global_active_power'].rolling(window).mean()
df_hourly['rolling_std'] = df_hourly['Global_active_power'].rolling(window).std()
df_hourly.dropna(inplace=True)
df_hourly.head()
# -------------------------------------------------------
# Z-Score Calculation
# -------------------------------------------------------
df_hourly['z_score'] = (
    (df_hourly['Global_active_power'] - df_hourly['rolling_mean']) /
    df_hourly['rolling_std']
)
df_hourly.head()
# -------------------------------------------------------
# Detect Anomalies
# -------------------------------------------------------
threshold = 3
df_hourly['z_anomaly'] = np.where(abs(df_hourly['z_score']) > threshold, 1, 0)
print("Number of anomalies detected:", df_hourly['z_anomaly'].sum())
# -------------------------------------------------------
# Plot Z-Score Anomalies
# -------------------------------------------------------
plt.figure(figsize=(15,5))
```

```
plt.plot(df_hourly.index, df_hourly['Global_active_power'], label='Energy Usage')
anomalies = df_hourly[df_hourly['z_anomaly'] == 1]
plt.scatter(anomalies.index,
            anomalies['Global_active_power'],
            color='red',
            label='Anomaly')
plt.title("Anomaly Detection using Rolling Z-Score")
plt.xlabel("Time")
plt.ylabel("Global Active Power (kW)")
plt.legend()
plt.show()
from sklearn.ensemble import IsolationForest
# -------------------------------------------------------
# Features for Isolation Forest
# -------------------------------------------------------
iso_features = df_hourly[['Global_active_power',
                          'Voltage',
                          'Global_intensity',
                          'Sub_metering_1',
                          'Sub_metering_2',
                          'Sub_metering_3']]

iso_features.head()
# -------------------------------------------------------
# Train Isolation Forest
# -------------------------------------------------------
iso_model = IsolationForest(
    n_estimators=200,
    contamination=0.01,
    random_state=42
```

```
)
df_hourly['iso_anomaly'] = iso_model.fit_predict(iso_features)
df_hourly['iso_anomaly'] = df_hourly['iso_anomaly'].map({1:0, -1:1})
print("Isolation Forest anomalies:", df_hourly['iso_anomaly'].sum())
plt.figure(figsize=(15,5))
plt.plot(df_hourly.index, df_hourly['Global_active_power'], label='Energy Usage')
iso_anomalies = df_hourly[df_hourly['iso_anomaly'] == 1]
plt.scatter(iso_anomalies.index,
        iso_anomalies['Global_active_power'],
        color='purple',
        label='Isolation Forest Anomaly')
plt.title("Anomaly Detection using Isolation Forest")
plt.xlabel("Time")
plt.ylabel("Global Active Power (kW)")
plt.legend()

plt.show()
# -------------------------------------------------------
# Prophet Residual Calculation
# -------------------------------------------------------
prophet_results = forecast.set_index('ds')
df_hourly['prophet_pred'] = prophet_results.loc[df_hourly.index]['yhat']
df_hourly['residual'] = df_hourly['Global_active_power'] - df_hourly['prophet_pred']
df_hourly.head()
# -------------------------------------------------------
# Prophet Residual Z-Score
# -------------------------------------------------------
residual_mean = df_hourly['residual'].mean()
residual_std = df_hourly['residual'].std()
df_hourly['residual_zscore'] = (
```

```
    (df_hourly['residual'] - residual_mean) / residual_std
)
threshold = 3
df_hourly['prophet_anomaly'] = np.where(abs(df_hourly['residual_zscore']) > threshold, 1, 0)
print("Prophet anomalies:", df_hourly['prophet_anomaly'].sum())
plt.figure(figsize=(15,5))
plt.plot(df_hourly.index, df_hourly['Global_active_power'], label='Energy Usage')
prophet_anomalies = df_hourly[df_hourly['prophet_anomaly'] == 1]
plt.scatter(prophet_anomalies.index,
            prophet_anomalies['Global_active_power'],
            color='orange',
            label='Prophet Residual Anomaly')
plt.title("Anomaly Detection using Prophet Residuals")
plt.xlabel("Time")
plt.ylabel("Global Active Power (kW)")
plt.legend()
plt.show()
# -------------------------------------------------------
# Anomaly Detection Comparison
# -------------------------------------------------------
methods = ['Z-Score','Isolation Forest','Prophet Residual']
counts = [273,342,413]
plt.figure(figsize=(8,5))
plt.bar(methods, counts)
plt.title("Comparison of Anomaly Detection Methods")
plt.ylabel("Number of Detected Anomalies")
plt.show()
```